Autenticación en APIs

Autenticación es el proceso mediante el que validamos una identidad en una comunicación cliente-servidor.

Cuando creamos un API, es muy importante limitar el uso del mismo por cuestiones de seguridad. No cualquier persona con acceso a internet debería ser capaz de utilizar todos o algunos de los endpoints.

> Imaginemos que trabajamos en el equipo de desarrollo de un supermercado. Si creamos un API para manejar las compras de productos, solamente debería estar abierto al uso de los clientes, administradores, y vendedores.
>
> Terceros no deberían ser capaces de accesar el API o secciones de este, ya que podrían manipular los datos de formas que no deberían ser permitidas.
>
> Por esta razón, muchas páginas y servicios web utilizan autenticación, que comunmente la vemos aplicada como autenticación de `username/email` y `password`.



Ref. https://securityintelligence.com/wp-content/uploads/2018/10/si-advanced-authentication-feature.jpg

Usualmente el flujo del proceso de autenticación es el siguiente:

1. El cliente inicia la sesión de alguna forma, generalmente con su usuario y contraseña.

2. El servidor valida el usuario y la contraseña y retorna algún tipo de “llave” de autenticación.
    1. A partir de acá, el cliente utiliza esta “llave” para los demás requests.
    2. Esta llave suele ser un string encriptado de alguna forma.
3. El cliente hace una nueva solicitud al servidor, pero esta vez envía la “llave” en lugar de enviar los datos de usuario y contraseña.
4. El servidor verifica que la “llave” sea válida a partir de diferentes protocolos, como el formato de encriptado que tiene y los datos que retorna al desencriptarla si es posible.
    1. Si la “llave” es válida, se ejecuta la solicitud
    2. Si la “llave” no es válida, se retorna un mensaje de error correspondiente.

Sin embargo, a pesar de que para el usuario mostramos esta capa de seguridad como un simple login, por debajo de esto existen múltiples métodos de autenticación. A continuación abarcaremos los más comunes:

Basic Auth

Basic Auth es la autenticación más simple que podemos tener en un API. Para este caso, agregamos un nuevo `header` a los `headers` del request, y enviamos el par de datos `username:password` en formato `base64` con la palabra `Basic` antes de los datos, de la siguiente forma:

Copiar

```
{
  "Content-Type": "application/json",
  "Authorization": "Basic dXNlcm5hbWU6cGFzc3dvcmQ="
}
```

Comunmente, este valor se almacena en el header `Authorization`.

Si decodificamos el dato `dXNlcm5hbWU6cGFzc3dvcmQ=` de `base64` a texto plano, el valor decodificado resulta en `username:password`. Siempre estos valores van separados por un `:`

Este es el tipo de autenticación más básica y menos recomendada, ya que estamos poniendo en tránsito valores altamente sensibles, como lo son el usuario y la contraseña del servicio que estemos utilizando, lo que nos pone en riesgo de robo de datos.

Por esta razón es poco recomendable implementar este tipo de autenticación en nuestra aplicación.

Autenticación por API Key

La autenticación por API Key generalmente es implementada como un tipo de autenticación aparte a la del usuario para iniciar sesión.

Esta es comúnmente utilizada cuando tratamos de accesar un servicio directamente, casi siempre directo al API.

Un API Key consiste en un valor generado por el sistema que funciona literalmente como una llave. Esta puede tener expiración o puede funcionar infinitamente, así como puede tener límites de acceso para permitirle a los consumidores el uso solamente de ciertas secciones del sistema.

> Un API Key suele ser un string con caracteres alfadecimales, generado automaticamente por el servicio. Este key se almacena en la base de datos y luego se valida cuando el usuario trata de consultar el servicio con este valor en los headers. Por ejemplo:

Copiar

```
{
  "Content-Type": "application/json",
  "x-api-key": "1234567890abcdef"
}
```

En este caso, el API Key se almacena en un `header` cuyo nombre puede variar, ya que para un mismo request puede existir el caso donde se requieran múltiples API Keys. Si nuestro sistema se llama `Lyfter`, nuestro `header` podría llamarse `lyfter-api-key`.

Este tipo de autenticación es muy útil ya que permite generar y administrar manualmente las llaves que damos para el acceso al sistema. Brinda un muy buen nivel de observabilidad ya que toda operación realizada siempre va a estar ligada a un API Key, y además es eficiente ya que no requiere el uso de un nombre de usuario y una contraseña.

Sin embargo, es una autenticación poco común para páginas web que requieran un login, ya que no es muy amigable con los usuarios que no sean desarrolladores.

Autenticación por JWT Token

La autenticación por JWT Token (Json Web Token) es una de las más utilizadas por su facilidad de implementación, pero también por su nivel de seguridad.

Este tipo de autenticación genera un “token”, que es un valor encriptado, que contiene datos relevantes del usuario, como su nombre o su ID.

Un JWT Token tiene la siguiente apariencia:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Copiar

También, el token tiene varias partes que las señalamos con colores:

Header: Almacena el tipo de algoritmo con el que fue encriptado, para realizar la verificación más adelante.

Payload: Son los datos del usuario, donde podemos agregar o quitar como sea necesario.

Firma de verificación: Esta parte almacena los datos del header y el payload encriptados, junto con una firma adicional agregada por nosotros que debe ser **secreta** ya que sin este valor los datos no se pueden desencriptar. Esta es la capa de seguridad de este tipo de autenticación.

```
{
  "alg": "HS256",
  "typ": "JWT"
}

{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}

HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
   mysignature
)
```

Copiar

Para usar esta autenticación, también la incluimos en los headers del request. Usualmente se envía en el header `Authorization` con la palabra `Bearer` antes del valor:

```
{
  "Content-Type": "application/json",
  "Authorization": "Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3
ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.Sf
lKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c"
}
```

Copiar

Autenticación OAuth 2.0

OAuth 2.0 es el estándar de autenticación actual de la industria. Es más complejo que los demás pero a su vez permite realizar niveles de autenticación que otros métodos no soportan. Por su complejidad, muchas veces se prefiere utilizar otro tipo de autenticación como JWT Token.

OAuth 2.0 funciona similar a un API Key o un Token, de forma que no exponen las credenciales de acceso a una plataforma.

Veamos el siguiente ejemplo:

Estamos utilizando una plataforma de almacenamiento de fotografias llamada LyftPhoto.

Queremos imprimir estas fotos pero nuestro servicio no lo permite, entonces encontramos otro llamado LyftPrint.

LyftPrint nos pide acceso a la plataforma LyftPhoto para obtener las fotos que se van a imprimir, por lo que nos salta una pantalla para permitir este acceso.

> Cuando LyftPrint solicita este acceso, le envía a LyftPhoto un código para identificar al usuario y un nivel de acceso que necesita para la cuenta. En este caso solo necesita permisos para ver las fotos, entonces no requiere modificarlas ni eliminarlas.
>
> Si LyftPhoto aprueba el acceso, la plataforma le retornará a LyftPrint un token de acceso con los permisos limitados para lo que requiere.

A partir de acá, cualquier autenticación que necesite LyftPrint para accesar a LyftPhoto será realizada a través de este token de OAuth 2.0 que fue generado.

A nivel de request, este token se envía de la misma forma que un `Bearer token` como el del ejemplo pasado:

Copiar

```
{
  "Content-Type": "application/json",
  "Authorization": "Bearer <OAuth2.0 Token>"
}
```

Ahora, conociendo la teoría, avancemos a la siguiente lección donde vamos a implementar autenticación en el backend de tipo JWT Token.